

Aula 06: Subprogramação I

Funções, reuso de código e módulos

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Revisitando as aulas anteriores

- ⌋ Nas aulas anteriores começamos a construir alguns programas em Python
 - * Tratamos as instruções de entrada e saída de dados
 - * Estrutura de um programa Python
 - * Estruturas de controle (desvio condicional e repetição)
- ⌋ À medida que um programa cresce, é comum que certos trechos de código sejam repetidos uma vez que o programa é uma sequência de comandos de máquina
- ⌋ Nesta aula, vamos aprender sobre um conceito importante do ponto de vista da estrutura e manutenção de sistemas: a **subprogramação**

- ⌈ Quando desenvolvemos um programa de computador, o código-fonte tende a crescer de acordo com as funcionalidades inseridas
 - * Se o código cresce, é provável que determinadas tarefas acabem se repetindo em diferentes trechos do código-fonte
- ⌈ Com os computadores modernos, copiar e colar um trecho do código-fonte não chega a ser uma tarefa muito aborrecida, porém é ineficiente
 - * Se o trecho replicado possuir um erro, como fica a correção do código?
 - * Se o código todo se concentra no corpo do programa principal, o quão legível é aquele código?

- ⌋ Este tipo de replicação pode estar presente até mesmo em questão de tarefas dependentes
- ⌋ Este comportamento também é observável em definições matemáticas
 - * Um **fatorial** de n é o produto dos n primeiros naturais
 - * Um **arranjo simples** é o quociente de um **fatorial** por outro **fatorial**
 - * A **média** de um conjunto de valores é o quociente da soma dos valores pelo tamanho do conjunto
 - * A **variância** é uma medida estatística cujo cálculo depende da **média**
 - * O **desvio padrão** é uma medida estatística cujo valor é igual à raiz quadrada da **variância**

- ⌋ Além de questões de manutenção e dependência de tarefas, precisamos nos lembrar que um código-fonte “grande” implica em um programa “grande”
 - * Nem todas as instruções do programa estão presentes no código-fonte
 - * Código-fonte maior implica número maior de instruções
 - * Número maior de instruções implica maior necessidade de memória para armazenar e executar o programa
- ⌋ Embora os computadores atuais disponham de boa oferta de memória em custo/benefício, usar mais memória que o necessário compromete a usabilidade de um programa

- ⌋ **Subprogramação** é a técnica de dividir o código-fonte de um programa em partes menores e que executam tarefas específicas
 - * Esses trechos podem ser reutilizados dentro de um mesmo programa ou em outros programas
 - * Ao isolar esses trechos, a manutenção de possíveis erros acaba sendo simplificada
 - * Princípio da técnica de **modularização**

- ⌋ **Modularização** é o conceito de arquitetura em que o sistema é composto por módulos independentes que operam de maneira cooperativa por meio de uma interface

- ☾ Os subprogramas costumam ser de dois tipos
 - * **Funções** são subprogramas que dão sempre um dado como resposta
 - * **Procedimentos** são subprogramas que executam tarefas e não retornam nenhum dado
- ☾ Em Python, todos os subprogramas são chamados **funções**
 - * Devido à tipagem dinâmica, as funções em Python podem ter retornos de tipos de dados diferentes
 - * Um dos retornos possível é **None**, que é um valor especial no Python que representa o vazio, o nulo

```
def nome_da_função(parâmetros)
```

- ⌋ `def` é uma palavra reservada de Python para declarar uma função, i.e., um bloco de código que pode ser reutilizado
- ⌋ `nome_da_função` segue as mesmas regras de nomenclatura de uma variável
- ⌋ `parâmetros` é uma listagem que indica quantos valores de entrada terá a função
- ⌋ Os parâmetros sempre são representados por identificadores semelhantes aos das variáveis

Funções em Python

```
def nome_da_função(parâmetros):  
    < bloco de comandos >  
    return < saída >
```

- ⌞ A tabulação define quais comandos devem ser executados quando a função for chamada
- ⌞ Ao chamarmos uma função, a execução é desviada para o bloco de comandos da função
- ⌞ A lista de parâmetros é separada por vírgulas

Exemplo de função

﴿ Observe a função fatorial a seguir

```
def fatorial(x):  
    fat = 1  
    for i in range(1,x+1):  
        fat *= i  
    return fat
```

- * x é o único parâmetro de entrada
- * Pela natureza do comando, vemos que x deve ser um número inteiro
- * A função tem um retorno único, a variável **fat** que também é um inteiro
- * Podemos incluir anotações no código para indicar os tipos esperados dos dados das funções

Dicas de tipos

☾ As **dicas de tipos** (*type hints*) são anotações inseridas nas funções que permitem que IDE's avisem o programador sobre potenciais erros de tipagem

- * Usamos : nas anotações dos parâmetros
- * Usamos -> na anotação do retorno da função
- * Se algum parâmetro ou retorno pode ser de mais de um tipo, separamos os tipos com |
- * Quando usamos coleções, colocamos o nome da coleção e os tipos de dados da coleção entre colchetes se preciso

```
def fatorial(x: int) -> int :  
    fat = 1  
    for i in range(1,x+1): fat *= i  
    return fat
```

Exemplos de dicas de tipos

```
def fatorial(x: int) -> int | None :  
    if x < 0 : return None  
    fat = 1  
    for i in range(1,x+1):  
        fat *= i  
    return fat
```

```
def comb_simples(n: int , p: int) -> int | None :  
    if n < 0 or p < 0 : return None  
    if p > n : return None  
    denominador = fatorial(p) * fatorial(n-p)  
    return fatorial(n) // denominador
```

Exemplos de dicas de tipos

- ☞ Como as funções abaixo só retornam valores não-negativos, poderíamos usar -1 como sinal de erro

```
def fatorial(x: int) -> int :  
    if x < 0 : return -1  
    fat = 1  
    for i in range(1,x+1):  
        fat *= i  
    return fat
```

```
def comb_simples(n: int , p: int) -> int :  
    if n < 0 or p < 0 : return -1  
    if p > n : return -1  
    denominador = fatorial(p) * fatorial(n-p)  
    return fatorial(n) // denominador
```

Considerações sobre funções

- ⌋ As funções que serão utilizadas no corpo do programa principal devem ser definidas antes dos comandos dele, para que assim possam ser chamadas para executar
- ⌋ Todo programa pode ter 0 ou mais funções
- ⌋ Toda função pode ser chamada 0 ou mais vezes
- ⌋ Uma função só é executada quando chamada
- ⌋ Uma função é algo com comportamento previsível, i.e., uma função não pode dar um resultado diferente se passarmos os mesmos parâmetros como entrada

☾ Variáveis locais:

- * Declaradas dentro de uma função
- * São vistas unicamente dentro da função
- * São criadas quando a função é chamada e deixam de existir quando a função é encerrada
- * Parâmetros de uma função possuem comportamento similar ao de uma variável local

☾ Variáveis globais:

- * Declaradas no corpo do programa principal (fora de qualquer função)
- * São visíveis e acessíveis por qualquer função do programa e também pelo corpo principal

Exemplo de escopo de variáveis

﴿ Considere o seguinte trecho de código

```
def fatorial(x: int) -> int :  
    if x < 0 : return -1  
    fat = 1  
    for i in range(1,x+1):  
        fat *= i  
    return fat  
  
if __name__ == "__main__":  
    a, b = 5, 2  
    c = fatorial(5) / fatorial(2)
```

Exercício: Diga que são as variáveis locais, variáveis globais e os parâmetros presentes no código.

Exemplo de escopo de variáveis

☞ Considere o seguinte trecho de código

```
def fatorial(x: int) -> int :  
    if x < 0 : return -1  
    fat = 1  
    for i in range(1,x+1):  
        fat *= i  
    return fat  
  
if __name__ == "__main__":  
    a, b = 5, 2  
    c = fatorial(5) / fatorial(2)
```

Exercício: Diga que são as variáveis locais, variáveis globais e os parâmetros presentes no código.

Resposta: As variáveis locais são fat e i. As variáveis globais são a, b e c. O único parâmetro usado é x.

Passagem de parâmetros

- ⌋ Quando uma função é chamada, precisamos fornecer os valores para os seus parâmetros, i.e., os dados de entrada que serão processados
 - * Isto pode ser feito informando diretamente os valores ou passando variáveis como parâmetros
 - * Os parâmetros na declaração da função são chamados **parâmetros formais**
 - * Os parâmetros na chamada da função são chamados **parâmetros reais**
- ⌋ Python utiliza um modelo de passagem de parâmetro chamado **referência de objeto** (*Call-By-Sharing*)

Referência de objeto

- ⌋ A função não recebe uma cópia do valor e nem uma referência da variável, mas sim um ponteiro direto para o objeto na memória
 - * Isso implica comportamentos distintos para objetos mutáveis e imutáveis
- ⌋ Objetos imutáveis (números, *strings*, tuplas) não podem ser modificados internamente, logo qualquer alteração cria um novo objeto local
- ⌋ Objetos mutáveis (listas, dicionários, conjuntos) podem ter seu conteúdo interno modificado diretamente, afetando o programa principal

Parâmetros default

- ⌋ Python permite que você defina valores prévios para argumentos de funções
 - * Se a função é chamada com menos argumentos que o definido, os parâmetros faltantes são completados com os valores *default*
 - * Parâmetros com valores padrão devem obrigatoriamente ficar ao final da assinatura da função, depois de todos os parâmetros obrigatórios
- ⌋ Para definir um valor padrão, basta usar o operador de atribuição (=) diretamente na assinatura da função
 - * A atribuição tem que ser escrita após a dica de tipo

Parâmetros default

☞ Vamos usar um exemplo da Física, o movimento retilíneo uniforme (MRU) e o uniformemente variado (MRUV)

✱ Qual a diferença dos dois?

✱ No MRU a aceleração é zero

```
def movRetilíneo(  
    t: int ,  
    s0: float ,  
    v0: float ,  
    a: float = 0.0  
) -> float :  
    return (s0 + (v0 * t) + (0.5 * a * t**2))
```

A forma acima também é aceita na hora de
escrever códigos fonte em Python

Módulos em Python

- ☾ Em Python, um **módulo** é simplesmente um arquivo com a extensão `.py` contendo códigos (funções, classes ou variáveis) que você pode reaproveitar em outros arquivos
 - * Eles servem para organizar o projeto e evitar a repetição de código
- ☾ O ecossistema do Python divide os módulos em três categorias
 - * **Módulos do Próprio Arquivo:** Criados por você no projeto
 - * **Biblioteca Padrão (Built-ins):** Já vêm instalados com o Python, como *math*, *datetime*, *random*, *os*, *sys*
 - * **Módulos de Terceiros:** Instalados via gerenciador de pacotes (*pip*), como *requests*, *pandas*, *pydantic*, *numpy*

Exemplos de módulos

⌋ **math** - funções matemáticas em números reais

✱ **sqrt(x)** - raiz quadrada de x

✱ **sin(x)**, **cos(x)**, **tan(x)** - seno, cosseno e tangente de x

✱ **factorial(x)** - fatorial de x

✱ **log(x,b)** - logaritmo de x na base b ($b = e$ por padrão)

⌋ **random** - gerar números pseudoaleatórios e realizar escolhas aleatórias em coleções

✱ **uniform(a,b)** - Retorna um número real $a \leq x < b$

✱ **randint(a,b)** - Retorna um número inteiro $a \leq x \leq b$

✱ **choice(lista)** - Seleciona e retorna um único item de uma lista

✱ **sample(lista,k)** - Seleciona e retorna k elementos distintos de uma lista

⌋ **Importação direta** - `import <modulo>`

- * Importa o módulo inteiro
- * Para usar qualquer função, você é obrigado a usar o nome do módulo como prefixo
- * Evita que funções do módulo apaguem ou criem conflito com funções que você já escreveu
- * O código pode ficar mais longo para digitar

⌋ **Importação com apelido** - `import <modulo> as <alias>`

- * Importa o módulo alterando o nome dele no seu arquivo
- * Muito utilizada em Ciência de Dados
- * Abrevia nomes de módulos muito longos

⌋ **Importação específica** - `from <modulo> import <função>`

- * Traz apenas a função ou variável específica que você precisa diretamente para o seu arquivo
- * O código fica mais limpo e você não precisa usar o prefixo do módulo
- * Se você já tiver uma função com o mesmo nome no seu arquivo, uma vai apagar a outra

⌋ **Importação geral** - `from <modulo> import *`

- * Importa absolutamente tudo o que existe dentro do módulo de uma vez só, sem precisar de prefixo
- * Você “polui” seu código com centenas de funções ocultas, não sabe de onde cada função veio e aumenta drasticamente a chance de *bugs* por conflito de nomes

Apelidos no Python

- ⌋ A palavra reservada **as** do Python serve para renomearmos localmente módulos, funções e variáveis/constantes que importamos para o nosso código-fonte

```
from <módulo> import <função> as <alias>  
# Renomeando uma função localmente
```

```
from <módulo> import <variável> as <alias>  
# Renomeando uma variável/constante localmente
```

```
import <módulo> as <alias>  
# Renomeando um módulo localmente
```

- ⌋ Desta forma, vinculamos localmente o que foi importado a um aliás (*nickname*, apelido)

